‘Enki’ Model Current State V1 (Implemented)

Basics:

$$s_{bt} = m + a_b + y_t + \epsilon_{bt} \quad (1)$$

Terms	
s_{bt}	Party vote share of vote in booth b at timestep t [can be represented as $s(b,t)$]
m	Vote-weighted average of booth baselines across elections (latent electorate-wide support)
a_b	Booth deviation from avg booth (natural local support of the booth)
y_t	Seat ‘tide’ term – relates to the vote pull or push experienced due to local, regional or statewide shifts and trends. Calculated later on through opinion polls. This term is meant to represent non-jitter, non-latent factors.
ϵ_{bt}	Epsilon term meant to describe a (temporary, non-latent, non-tide) local shift in vote share. This can be to describe the jitter present on poll day (sampling error of the booth) or can be used to tune the model to reflect the influence and feedback to canvassers and others. Bit of a wildcard term.

Note to reader: local ≠ electorate, local = booth, electorate ⊂ region ⊂ state

State-swing term:

$$y_t = \beta S_t + d_t \quad (2)$$

Terms	
y_t	Seat ‘tide’ at a given election (change due to non-latent factors)
β	State-Tide Sensitivity coefficient (how sensitive a given seat is to the determined statewide tides/changes).
S_t	Statewide share for same party at election t
d_t	Electorate specific tide (the part that the states’ election % influence doesn’t explain – this is just a local quirk of how a specific electorate perceives the state government)

Beta/Sensitivity Definition:

$$\beta = p_g + p_r + \mathrm{p_L} \quad (3)$$
$$p_g \approx 1$$

Terms	
β	State-Tide Sensitivity coefficient (how sensitive a given seat is to the determined statewide tides/changes)
p_g	Sensitivity due to global (statewide) factors. Should equal near to 1 (on parity with swings measured in polling data)
p_r	Sensitivity due to regional factors (measured amongst nearby seats)
p_L	Sensitivity due to local factors (this term I anticipate being very weak i.e., $p_L \approx 0$ in a large city like Sydney)

Note for implementation: constraint/regularisation may be required for p_g, p_r

All together:

$$s(b,t) = m + a_b + (p_g + p_r + \mathrm{p_L})S_t + d_t + \epsilon_{bt} \quad (4)$$

For the purposes of describing the model, this equation can be broken very vaguely into the following:

$$LATENT = m + a_b$$
$$TIDE = (p_g + p_r + \mathrm{p_L})S_t + d_t$$
$$LOCAL = \epsilon_{bt}$$

Given that VS means vote share, we can surmise the 'Enki' model descriptively as:

$$BOOTH_{VS} = LATENT_{VS} + TIDE_{VS} + LOCAL_{VS}$$

$$s(b,t) = m + a_b + (\beta S_t + d_t) + \epsilon_{bt}$$

'Enki' Model Changes -> V2

Gets a bit more interesting -> these are just the changes to the above that have been made in V1. The assumption is that V1 carries onto V2 just with the following additions.

Linear Scale -> Logit Scale:

$$s_{bt} = m + a_b + y_t + \epsilon_{bt}$$

This equation (equation 1) is a purely linear addition and given this, swings in y_t can cause the s_{bt} to go above or below 0. Yes, this is purely an edge case, but the solution was to move over to a $logit$ scale, meaning that vote shares always remain $0 \leq x \leq 1$

$$logit(p) = \log(\frac{p}{1-p})$$

Just to establish how this scale works, for an 80% vote share the following occurs:

$$80\%_{vs} \rightarrow logit(0.8) = \log(4) \approx 1.38$$

And the inverse logit means that whatever value is inputted is bound by $0 \leq x \leq 1$:

$$expit(z) = \frac{1}{1+e^{-z}} \rightarrow 0 \leq expit(z) \leq 1$$

This also fixes another problem, around the state swing alignment – the Problem before was that statewide swings were measured as deviation variables such that $\Delta S_t = S_t - S_{ref}$. Basically this means that the change in state vote share was relative to a single statewide election in absolute terms. This means that a suddenly high S_t could cause a severe artificial jump in ΔS_t based on statewide polling instead of local swings. The fix comes in the form of aligning the swing on the logit scale such that:

$$x_t = logit(S_t) - mean(logit(S))$$

Essentially this fixes two problems, the bounding problem (ensures that this sticks between 0 and 1) and also the swing alignment problem, by ensuring that statewide shift are centred as logit deviations to prevent absolute statewide polling from causing artificial jumps in local predictions.

β redefinition & minor perceptual changes to a_b, ϵ_{bt}

We had the formula before of $\beta = p_g + p_r + p_L$ but the problem comes in the fact that we don't have enough geographical degrees of freedom to assess each term

individually. I can't decompose the terms properly and having so many confounding/unestablished variables caused the later monte carlo analysis to have poor convergence and high r hat value (not good). We've got to control variables better than is being done here, and the only thing that can be assessed with any accuracy is the β term directly. This means that β is revised to:

$$\beta = p_g + p_r + p_L \rightarrow \beta \sim Normal(\beta_g, sd)$$

Yes, we are dropping the $p_r + p_L$ terms, but this does not mean that regional or local factors do not factor into the overall equation. I don't know by which ratio these three variables are represented (i.e., 95%, 3%, 2%) and since I can't decompose them any further, I can't determine what their values are. I noticed upon reflection we already have other representations of $p_r + p_L$ and so therefore concluded it's best to let booth level variations (some mix of p_r, p_L) be represented by a_b and any other local residual be absorbed by the term ϵ_{bt} just leaving the one term not represented anywhere else p_g – which is to be a normal distribution represent the global sensitivity β_g such that:

$$\beta \approx p_g \sim Normal(\beta_g, sd)$$

And for later, to marry the above to the later Bayesian probabilities:

$$N(Mean, Variance) \Leftrightarrow N(\beta_g, \sigma_\beta^2)$$

I know the title says changes to a_b, ϵ_{bt} but this is only half true – the better way to define this is that $p_r + p_L$ were assessed to be present elsewhere already and therefore a bit redundant in assessing β.

Least Squares to Binomial Counts (for vote-count)

The previous approach was to model s_{bt} using ordinary least squares regression which treats each booth as if it has the same statistical weight regardless of size. This is naturally incorrect, as the example of East Hills electorate shows where there is one booth, Sydney Town Hall that has 101 votes and is weighted the same as a booth with 2000 votes in the local region (Nonsensical!).

The change being made is simple – instead of treating it as discrete booth-units (i.e., here's 23 booths for you) it treats it as discrete 'success' counts such that v votes out of n_{bt} total formal votes. This is modelled, where p_{bt} is probability of booth b at time t quite simply as:

$$v_{bt} \sim Binomial(n_{bt}, p_{bt})$$

This means that smaller booths have higher statistical variance and naturally carry less weight than booths with thousands of votes. Essentially this just make sure that booths with higher vote counts are treated as more reliable in outcome than smaller booths.

Point Estimation -> Bayesian Estimation (THIS ONE IS MAJOR)

Likely the biggest change of this version is moving away from point estimation into Bayesian Markov Chain Monte Carlo (MCMC) estimation. Python makes this super easy with the PyMC package. The 'points' being estimated were m, a_b, β and having them as fixed points meant that there was no way to quantify uncertainty or sampling risks of these variables – we just accepted the output given as a 'fixed point'.

The change made, which is major, is to sample thousands of plausible parameter sets from the joint posterior distribution. The joint posterior distribution captures how multi-variables behave jointly and looks for correlations between them – essentially establishes given probabilities of operation based on existing data for each of the variables. This results in every parameter m, α_b, β, σ_d, σ_ϵ becoming a probability distribution rather than fixed static point estimations.

The new probability estimations are then run through an MCMC estimation (Enjoy googling the Markov Chain part of this). The Monte Carlo part of this is simply just using random samples to estimate mathematical quantities based on probability distributions (i.e., run 10 distributions present on parts of the model 10000 times and see the result… can range from low probability outcomes to high probability outcomes but the repetition means the most likely outcome triggers the most and a final probability curve is generated)

Given that $\mu_0 = logit(45)$, or the 45% of the statewide vote share won across the past elections, that means that the changes amount to:

$$m \sim N(\mu_0, \sigma_m^2)$$
$$\alpha_b \sim N(0, \sigma_\alpha^2)$$
$$\beta \sim N(\beta_g, \sigma_\beta^2)$$
$$\epsilon_{bt} \sim N(0, \sigma_\epsilon^2) \rightarrow \sigma_\epsilon \sim HalfNormal(\tau)$$

One additional fix due to this was a problem that booth baselines (a_b) were calculated independently in V1 via basic linear algebra meaning that booths that were short-lived (i.e., one or two elections only) were prone to extreme overfitting and erratic swings (really short datasets proved a serious problem). This change has meant that $\alpha_b \sim N(0, \sigma_\alpha^2)$, and this now means that the distributions handle the statistical shrinkage! Booths with low historical data are regularised toward the electorate wide mean, while booths with many election histories retain all their funky local quirks.

Monte Carlo Predictive Simulations

A quick note first, that this is different from the previous MCMC change mentioned before. What happened prior was that it was a simple $f(x_0, \dots, x_n) = y$ model, where you plugged in your values for $x_0, \dots, x_n$ and spat out a deterministic "worst, best, average" scenario.

The change now is that it simulates thousands of future elections via a Montecarlo simulation. The worst, best cases now reflect the 5th percentile, and the 95th percentile and the program outputs a plot of predicted outcomes instead of a fixed point. This really relies on the previous step – running probabilities as over and over again thousands of time to determine what the outcomes are.

Minor Changes Worth Mentioning

- For the d_t term in equation 2, the change is that $d_{next} \sim N(\rho d_{last}, \sigma_d)$ where ρ is the persistence factor for a specific electorate -i.e., how persistent trends carry forward into the next election
- Opinion Polling uncertainty (not part of the model, but relevant for gauging accuracy) is calculated with a house-effect variance buffer which is implemented as: $\sqrt{se_{poll}^2 + pollhousesd^2}$.
    - Poll house effect ($pollhousesd$) is a statistical buffer to handle real world polling inaccuracies and prevents the model from becoming overconfident
    - Used on polling data primarily, NOT the statistical model
    - se means standard error, and the model does not just use the opinion polls given standard error (se_{poll}) but instead inflates the variance by adding a house-effect standard deviation buffer.
    - In practical terms means that the Bayesian engine here projects a wider margin of uncertainty around the polls – this ensures that the final intervals account for the fact that the polls themselves can be systematically skewed.
    - This is naturally just $pollhousesd = 1$ but I intend on at a later point making this variate depending on how polling skews against all elections known
- Predictive draws can occasionally layer temporary local residual noise (σ_ϵ) into future booth simulations.

Vote Count from Static to Bayesian

As title says – prior it treated the voter baseline as stable, but this is incorrect with the example of increasing prepoll of the years point towards a problem around assuming that people voting in one place will remain static over time. The outcome of this change is to use a hierarchical negative binomial distribution

$$n_{i,t} \sim NegBin(\mu_{i,t}, \alpha_{disp})$$
$$\log(\mu_{i,t}$$

<Need to keep annotating, pls Alec>

Man I don't like annotating.